

РО 1.4. Основные сервисы Linux для предприятия

Лекция 9.

Linux-router, nftables, NAT, port forwarding и WireGuard VPN

Цель лекции

Понять, как Linux-сервер выполняет роль маршрутизатора между сетями

Объяснять назначение IP forwarding, таблицы маршрутизации и транзитного трафика

Настраивать базовые правила nftables, NAT masquerade и port forwarding

Различать SNAT, DNAT, policy drop и разрешающие правила firewall

Понимать назначение WireGuard VPN и выполнять базовую диагностику туннеля

Учебные вопросы: 1-4

1. Linux как маршрутизатор корпоративной сети
2. Архитектура современного брандмауэра nftables
3. Трансляция сетевых адресов NAT и masquerading
4. Проброс портов Port Forwarding и DNAT
5. Введение в VPN: объединение сетей и удаленный доступ
6. WireGuard VPN: архитектура и преимущества
7. Практика: настройка WireGuard туннеля маршрутизатор-клиент
8. Диагностика, мониторинг и типичные ошибки

Учебный сценарий лекции

gw01.company.test работает как Linux-router между внешней и внутренней сетью

WAN (Wide Area Network) — внешняя сеть; интерфейс enp0s3 использует учебный адрес 203.0.113.10

LAN (Local Area Network) — внутренняя сеть; enp0s8 имеет адрес 192.168.10.1/24, а web01 — 192.168.10.20

WireGuard-интерфейс wg0 использует VPN-сеть 10.8.0.0/24 для удаленного клиента

1. Linux как маршрутизатор корпоративной сети

Router, маршрутизатор, пересылает IP-пакеты между разными сетями

Linux-сервер может выполнять роль маршрутизатора, если у него есть несколько сетевых интерфейсов

В учебном стенде `enp0s3` смотрит во внешнюю сеть, а `enp0s8` смотрит в LAN

Маршрутизация нужна, чтобы узлы LAN могли выходить наружу и получать ответы

Интерфейсы и сети gw01

enp0s3 — внешний интерфейс, через который gw01 выходит в сторону провайдера или учебного Интернета

enp0s8 — внутренний интерфейс для сети 192.168.10.0/24

Адрес 203.0.113.10 относится к документационному диапазону и используется только как учебный внешний IP

Для клиентов LAN шлюзом по умолчанию должен быть 192.168.10.1

Транзитный пакет и `ip_forward`

Транзитный пакет не предназначен самому `gw01`, а должен пройти через него в другую сеть

По умолчанию Linux часто не пересылает такие пакеты между интерфейсами

`net.ipv4.ip_forward` — параметр ядра, который разрешает IPv4-маршрутизацию

Команда `sysctl net.ipv4.ip_forward` показывает текущее значение параметра

Включение маршрутизации IPv4

`sudo sysctl -w net.ipv4.ip_forward=1` включает пересылку IPv4-пакетов до перезагрузки

Строка `net.ipv4.ip_forward=1` в `/etc/sysctl.d/99-router.conf` делает настройку постоянной

`sudo sysctl --system` перечитывает системные `sysctl`-файлы

Если `ip_forward` равен 0, NAT и VPN могут подниматься, но транзитный трафик не пройдет

Таблица маршрутизации

Routing Table, таблица маршрутизации, определяет, через какой интерфейс отправлять пакет

Команда `ip route` показывает маршруты Linux-узла

Маршрут `default` указывает путь для сетей, которые не совпали с более точными маршрутами

На клиентах LAN default route должен указывать на 192.168.10.1

Путь пакета через Linux-router

Пакет от client01 приходит на внутренний интерфейс enp0s8

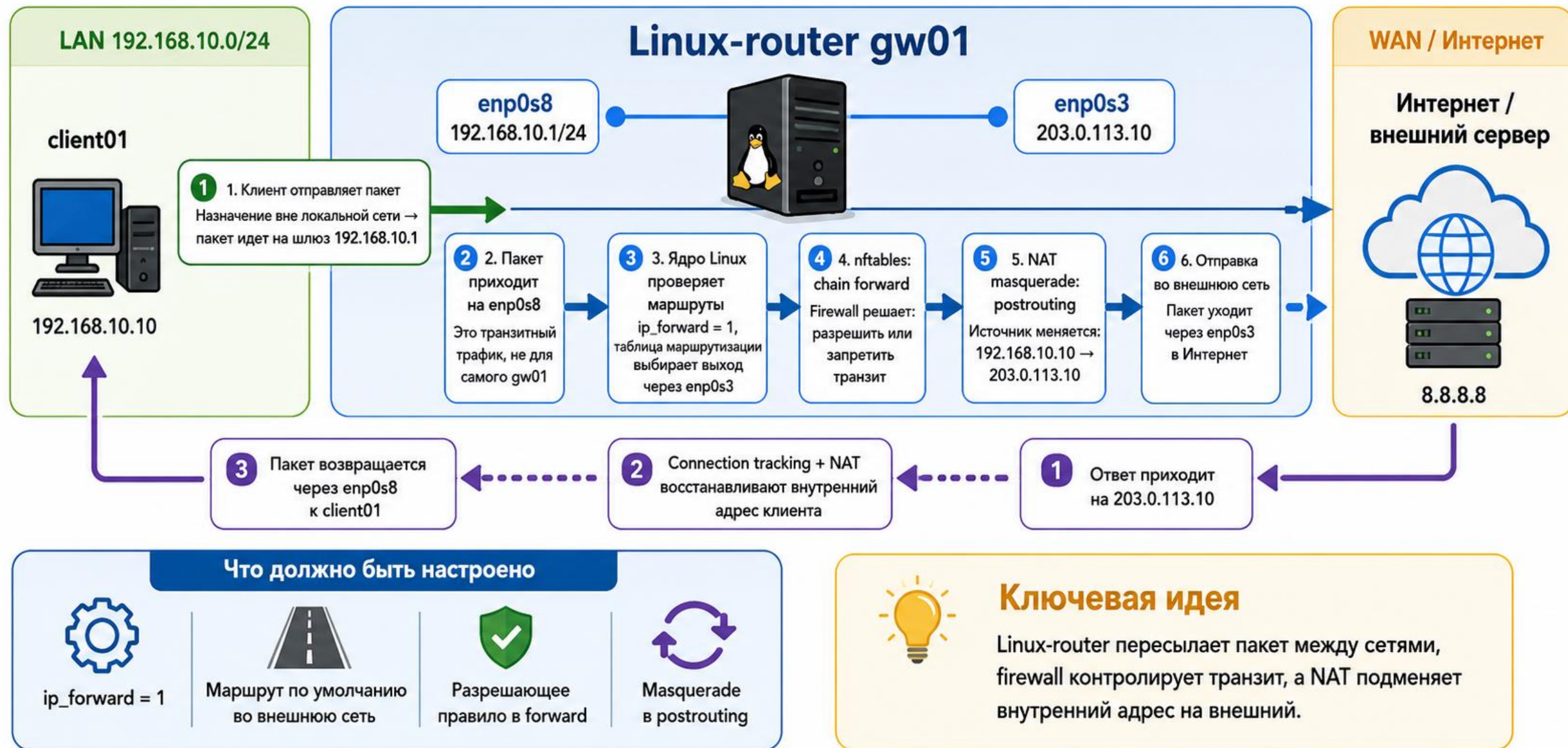
Ядро проверяет таблицу маршрутизации и выбирает исходящий интерфейс enp0s3

Firewall в цепочке forward решает, можно ли переслать этот пакет

NAT в postrouting может заменить адрес источника перед выходом во внешнюю сеть

Прохождение пакета через Linux-router

Как транзитный пакет проходит через маршрутизатор, firewall и NAT



2. Архитектура современного брандмауэра nftables

Firewall, брандмауэр, контролирует, какие пакеты разрешены или запрещены

nftables — современный фреймворк фильтрации пакетов в ядре Linux

nftables пришел на смену классическому iptables и объединяет IPv4, IPv6 и разные типы обработки

Администратор описывает таблицы, цепочки и правила, а ядро применяет их к пакетам

Family, table, chain и rule

Family, семейство, определяет область действия правил: ip, ip6 или inet

Table, таблица, является контейнером для цепочек

Chain, цепочка, содержит набор правил и может быть привязана к hook ядра

Rule, правило, описывает условие и действие: accept, drop, reject, masquerade или dn timer

Hooks в nftables

Hook, точка подключения, показывает, на каком этапе ядро передает пакет в цепочку

input обрабатывает пакеты, адресованные самому gw01

forward обрабатывает транзитные пакеты между интерфейсами

output обрабатывает пакеты, созданные самим gw01

prerouting и postrouting часто используются для NAT до и после выбора маршрута

Policy deny by default

Deny by default означает, что по умолчанию трафик запрещен, а нужное разрешается явно

Такой подход безопаснее, чем разрешить все и пытаться закрывать отдельные порты

Для маршрутизатора особенно важна цепочка forward, потому что через нее идет LAN, NAT и VPN

Перед включением policy drop нужно подготовить правила, чтобы не потерять удаленный доступ

Базовая filter-таблица

`sudo nft add table inet filter` создает таблицу фильтрации для IPv4 и IPv6

`sudo nft add chain inet filter input '{ type filter hook input priority 0; policy drop; }'` создает цепочку входа с запретом по умолчанию

`sudo nft add chain inet filter forward '{ type filter hook forward priority 0; policy drop; }'` создает цепочку транзитного трафика

`sudo nft add rule inet filter input ct state established,related accept` использует connection tracking и разрешает ответы

Разрешающие правила для gw01

```
sudo nft add rule inet filter input iifname "lo" accept
```

разрешает локальный loopback-трафик

```
sudo nft add rule inet filter input iifname "enp0s8" tcp dport
```

22 accept разрешает SSH (Secure Shell) с LAN

```
sudo nft add rule inet filter input iifname "enp0s3" udp dport
```

51820 accept разрешает WireGuard снаружи

```
sudo nft add rule inet filter forward ct state established,related
```

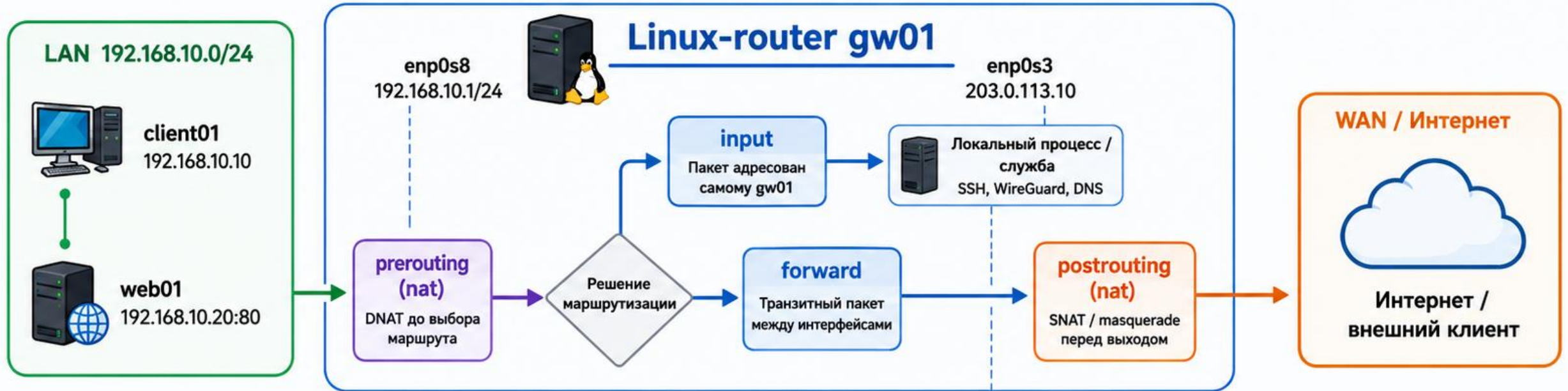
accept разрешает обратный транзитный трафик

```
sudo nft add rule inet filter forward iifname "enp0s8" oifname
```

"enp0s3" accept разрешает LAN выходить наружу

nftables hooks: input, forward и nat

Где Linux обрабатывает локальный, транзитный и NAT-трафик



1 input



- Обрабатывает пакеты, адресованные самому Linux-router
- Примеры: SSH на gw01, UDP 51820 для WireGuard
- Типичное действие: ассепт или drop

2 forward



- Обрабатывает транзитный трафик
- Пример: client01 → Интернет
- Именно здесь firewall решает, можно ли пересылать пакет дальше

3 nat



- prerouting: DNAT меняет адрес назначения до маршрутизации
- postrouting: masquerade меняет адрес источника перед выходом
- Примеры: port forwarding и выход LAN в Интернет



Сценарий А: Пакет к самому маршрутизатору

- 1 Клиент идет на 192.168.10.1:22
- 2 Пакет проходит через input
- 3 SSH-служба получает пакет



Сценарий В: Транзитный пакет в Интернет

- 1 client01 отправляет пакет наружу
- 2 Пакет проходит через forward
- 3 postrouting выполняет masquerade
- 4 Пакет уходит через enp0s3



Ключевая идея: input — трафик к самому роутеру, forward — транзитный трафик, nat — изменение адресов в prerouting и postrouting.

Просмотр и сохранение правил nftables

`sudo nft list ruleset` показывает весь активный набор правил nftables

`sudo nft -f /etc/nftables.conf` загружает правила из файла

`sudo systemctl enable nftables` включает загрузку правил при старте системы

Перед применением ruleset нужно иметь локальный доступ или резервную SSH-сессию

3. Трансляция сетевых адресов NAT и masquerading

NAT (Network Address Translation) — трансляция сетевых адресов в проходящих пакетах

NAT нужен, когда множество внутренних узлов используют один внешний адрес маршрутизатора

Внутренние адреса 192.168.10.0/24 не

маршрутизируются в публичном Интернете

gw01 должен заменить адрес источника клиента на свой внешний адрес перед выходом наружу

SNAT и masquerade

SNAT (Source NAT) изменяет адрес источника в исходящем пакете

Masquerade — вариант SNAT, при котором внешний адрес берется с исходящего интерфейса

Masquerade удобен, если внешний адрес может меняться

Для LAN-клиента ответ из внешней сети возвращается на gw01, а затем обратно во внутреннюю сеть

Настройка masquerade в nftables

`sudo nft add table ip nat` создает IPv4-таблицу NAT для учебного стенда

`sudo nft add chain ip nat postrouting '{ type nat hook postrouting priority srcnat; policy accept; }'` создает цепочку после маршрутизации

`sudo nft add rule ip nat postrouting oifname "enp0s3" masquerade` включает подмену источника на внешнем интерфейсе

`postrouting` используется потому, что исходящий интерфейс уже выбран таблицей маршрутизации

Проверка выхода LAN через NAT

На client01 команда `ip route` должна показывать default via 192.168.10.1

На gw01 команда `sysctl net.ipv4.ip_forward` должна вернуть значение 1

`sudo nft list ruleset` должен показывать правило masquerade в postrouting

ping внешнего адреса с client01 проверяет маршрутизацию и NAT без зависимости от DNS

NAT masquerade для выхода LAN наружу

Как Linux-router подменяет внутренний адрес и выпускает клиентов LAN в Интернет



Типовые ошибки NAT

`ip_forward` выключен, поэтому Linux не пересылает транзитные пакеты

В forward-цепочке `policy drop`, но нет разрешающего правила LAN-to-WAN

Masquerade указан не на том исходящем интерфейсе

У клиента LAN неверный gateway или маска сети

Проверка начинается с `ip route`, `sysctl`, `nft list ruleset` и `ping` по IP-адресу

4. Проброс портов Port Forwarding и DNAT

Port Forwarding, проброс портов, публикует внутреннюю службу через внешний адрес маршрутизатора

DNAT (Destination NAT) изменяет адрес или порт назначения во входящем пакете

В учебном примере внешний TCP (Transmission Control Protocol) порт 8080 на gw01 направляется на web01:80

Проброс портов требует не только DNAT, но и разрешения транзита в firewall

Когда используется DNAT

Внешний клиент подключается к 203.0.113.10:8080
gw01 получает пакет на внешнем интерфейсе enp0s3
DNAT меняет назначение пакета на 192.168.10.20:80
После этого пакет маршрутизируется во внутреннюю
сеть к web01

Правило DNAT в nftables

`sudo nft add chain ip nat prerouting '{ type nat hook prerouting priority dstnat; policy accept; }'` создает цепочку до маршрутизации

`sudo nft add rule ip nat prerouting iifname "enp0s3" tcp dport 8080 dnat to 192.168.10.20:80` направляет входящий TCP 8080 на web01

prerouting используется потому, что адрес назначения нужно изменить до выбора маршрута

Правило DNAT не заменяет правила фильтрации в forward-цепочке

Разрешение forward для web01

```
sudo nft add rule inet filter forward iifname "enp0s3"  
oifname "enp0s8" ip daddr 192.168.10.20 tcp dport 80
```

ассерт разрешает трафик к web01

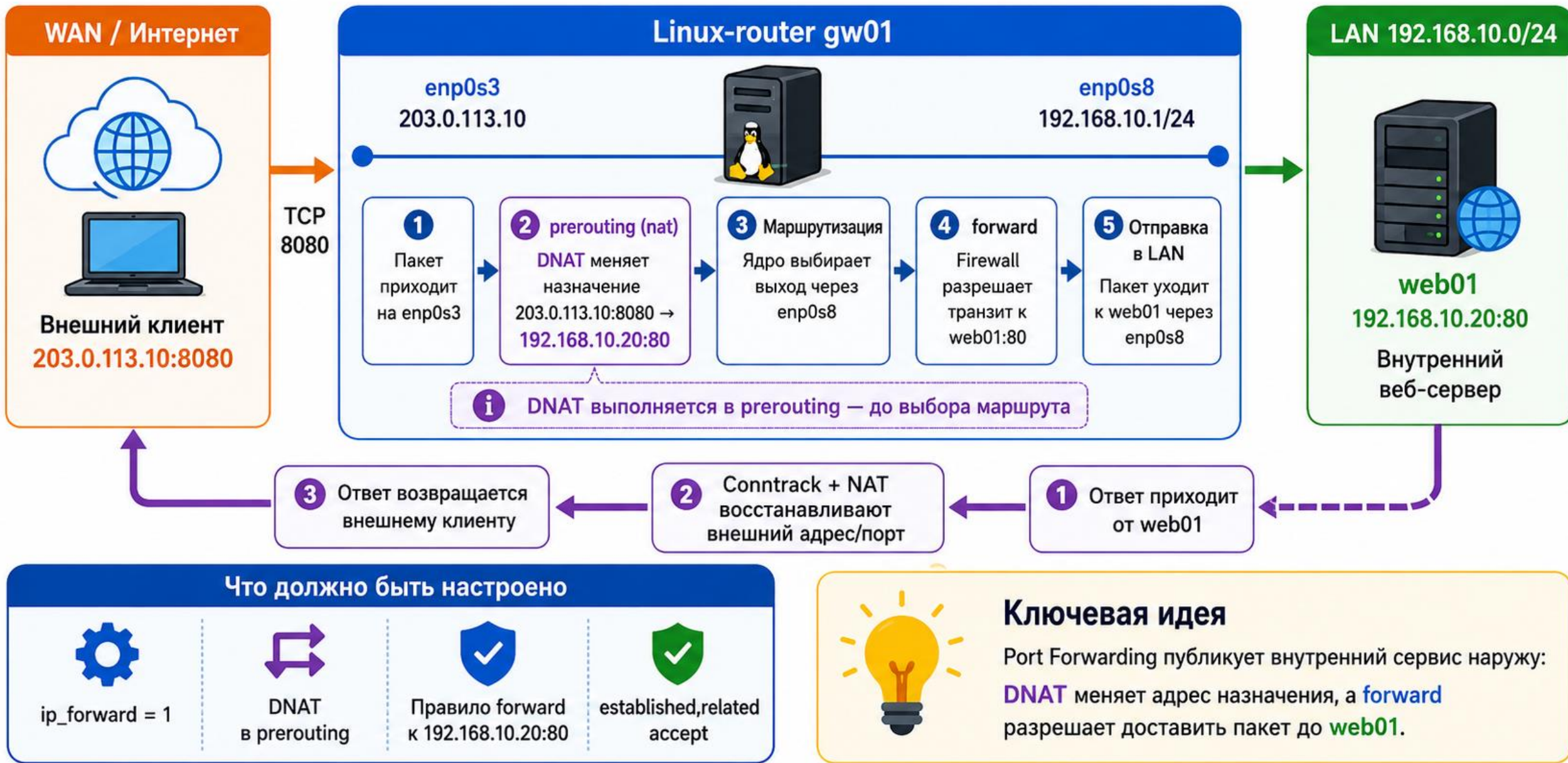
Правило должно быть достаточно узким: конкретный интерфейс, адрес и порт

Ответный трафик обычно разрешается правилом `state established,related` ассерт

Если DNAT есть, но forward запрещен, внешний клиент не получит ответ

Port Forwarding 203.0.113.10:8080 → web01:80

Как Linux-router публикует внутренний веб-сервер через DNAT и forward



Риски публикации внутреннего сервера

Проброс порта делает внутреннюю службу доступной снаружи

Нужно публиковать только необходимый порт, а не весь сервер

Служба web01 должна быть обновлена и защищена собственными настройками

Логи доступа к опубликованной службе нужно отправлять на централизованный log-сервер

Для административных сервисов лучше использовать VPN, а не прямой port forwarding

5. Введение в VPN: объединение сетей и удаленный доступ

VPN (Virtual Private Network) — виртуальная частная сеть поверх другой сети, обычно Интернета

VPN создает защищенный туннель между узлами или сетями

Через туннель можно передавать корпоративный трафик так, будто клиент находится внутри сети

VPN не отменяет маршрутизацию и firewall: для туннеля тоже нужны адреса, маршруты и правила

Site-to-Site и Remote Access

Site-to-Site VPN объединяет две сети, например офис и филиал

Remote Access VPN подключает отдельного удаленного сотрудника к корпоративной сети

В Site-to-Site важны маршруты между подсетями обоих офисов

В Remote Access важны учет клиента, доступные внутренние сети и безопасность устройства пользователя

Поколения VPN-решений

IPsec — мощный стандарт для site-to-site VPN, но его настройка может быть сложной

OpenVPN — распространенное решение, работающее поверх TLS и пользовательского пространства

WireGuard — современный минималистичный VPN, встроенный в ядро Linux

Выбор VPN зависит от совместимости, безопасности, скорости и требований организации

Адреса и маршруты в VPN

VPN-интерфейс получает собственный адрес,
например `wg0 = 10.8.0.1/24`

Удаленный клиент получает адрес внутри туннеля,
например `10.8.0.2/32`

Чтобы клиент видел LAN `192.168.10.0/24`, нужны
маршруты и разрешающие firewall-правила

Если маршрут есть только на одной стороне, связь
будет односторонней или полностью
неработоспособной

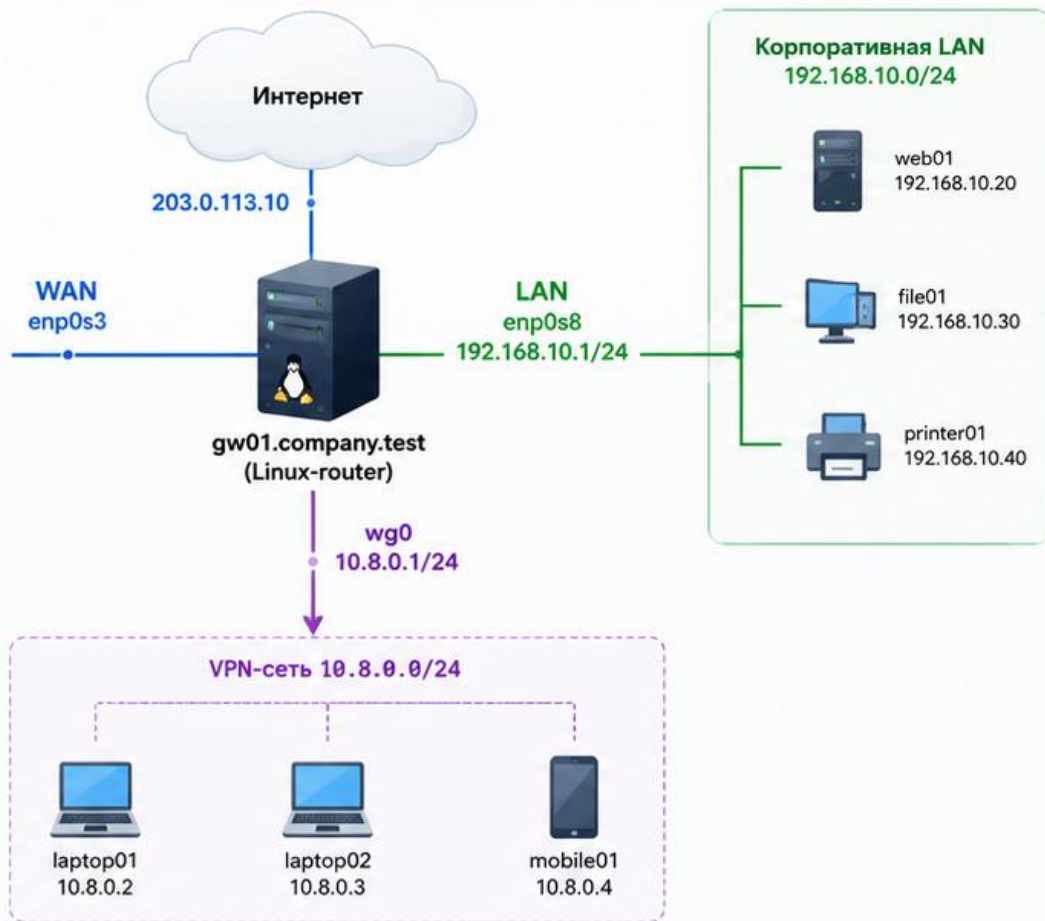
Remote Access VPN к корпоративной LAN

Безопасный удалённый доступ сотрудников к внутренней сети компании через WireGuard VPN

КАК ЭТО РАБОТАЕТ

-  Клиент подключается к VPN-серверу gw01 через Интернет.
-  Устанавливается зашифрованный WireGuard-туннель (wg0). Проверяется ключ клиента.
-  Клиент получает IP из VPN-сети 10.8.0.0/24 и маршрут до LAN 192.168.10.0/24 через gw01.
-  Трафик клиента идёт через туннель на gw01, затем маршрутизируется в корпоративную LAN.
-  Ответы из LAN возвращаются через gw01 в туннель и приходят к клиенту.

ПРИМЕР ПОДКЛЮЧЕНИЯ



ПРЕИМУЩЕСТВА

-  **Шифрование трафика**
WireGuard использует современные алгоритмы (Noise IK, ChaCha20, Poly1305).
-  **Простой удалённый доступ**
Сотрудники работают как будто в офисной сети, без проброса портов.
-  **Маршрутизация через VPN**
Доступ только к нужным сетям (например, 192.168.10.0/24).
-  **Высокая производительность**
Минимальные накладные расходы и быстрая установка соединения.

ПРИМЕР КОНФИГУРАЦИИ (КРАТКО)

Сервер (gw01) – /etc/wireguard/wg0.conf

```
[Interface]
Address = 10.8.0.1/24
ListenPort = 51820
PrivateKey = <server_private_key>

# Разрешить маршрутизацию и доступ в LAN
PostUp = sysctl -w net.ipv4.ip_forward=1
PostUp = nft add rule inet filter forward iifname "wg0" oifname "enp0s8" accept
PostUp = nft add rule inet filter forward iifname "enp0s8" oifname "wg0" ct
        state related,established accept
# Маршрут к LAN уже есть: 192.168.10.0/24 dev enp0s8
```

Клиент (laptop01) – /etc/wireguard/wg0.conf

```
[Interface]
Address = 10.8.0.2/24
PrivateKey = <client_private_key>
DNS = 192.168.10.1

[Peer]
PublicKey = <server_public_key>
Endpoint = 203.0.113.10:51820
AllowedIPs = 192.168.10.0/24, 10.8.0.0/24
PersistentKeepalive = 25
```

 AllowedIPs определяет, какие сети пойдут через VPN.

ЧТО НУЖНО НА GW01

- ✓ Включить маршрутизацию IPv4:
`sysctl -w net.ipv4.ip_forward=1`
- ✓ Разрешить форвард между wg0 и LAN (правила forward, как в примере выше)
- ✓ Открыть в firewall (nftables):
UDP порт 51820 для WireGuard
- ✓ Маршрут к LAN (192.168.10.0/24) должен быть в таблице маршрутизации

ПРОВЕРКА РАБОТЫ

```
# На клиенте
$ wg show
$ ping 10.8.0.1
$ traceroute 192.168.10.20

# На сервере
$ wg show
$ ip route | grep 10.8.0.
$ nft list ruleset | grep wg0

# пинг VPN-сервера
# пинг хоста в LAN
# маршрут должен идти через 10.8.0.1

# список подключенных пиров
# маршрут до VPN-сети
# правила firewall
```

БЕЗОПАСНОСТЬ

- ✓ Используйте уникальные ключи для каждого клиента.
- ✓ Разрешайте доступ только к необходимым сетям (AllowedIPs).
- ✓ Следите за журналами: `journalctl -u wg-quick@wg0`
- ✓ Регулярно обновляйте систему и проверяйте конфигурацию.

ИТОГ

WireGuard VPN позволяет сотрудникам безопасно подключаться к ресурсам корпоративной LAN из любой точки мира.

6. WireGuard VPN: архитектура и преимущества

WireGuard — современный VPN-протокол и реализация для Linux и других систем

WireGuard использует криптографические ключи вместо сложной инфраструктуры сертификатов

Он работает быстро, имеет небольшой код и хорошо подходит для серверов и мобильных клиентов

WireGuard использует UDP (User Datagram Protocol) и по умолчанию слушает порт 51820

Пирь и ключи WireGuard

Peer, пир, — участник WireGuard-туннеля: сервер или клиент

Каждый пир имеет private key и public key

Private key, закрытый ключ, хранится только на своем узле

Public key, открытый ключ, передается другой стороне и указывается в конфигурации peer

Cryptokey Routing

Cryptokey Routing — подход WireGuard, который связывает маршруты с открытыми ключами пиров
AllowedIPs определяет, какие адреса считаются принадлежащими конкретному peer

На сервере AllowedIPs = 10.8.0.2/32 означает адрес конкретного удаленного клиента

На клиенте AllowedIPs может включать 192.168.10.0/24, если клиент должен ходить в корпоративную LAN

WireGuard и firewall

На внешнем интерфейсе gw01 нужно разрешить UDP 51820

В forward-цепочке нужно разрешить трафик между wg0 и LAN

Если VPN-клиент должен выходить в Интернет через gw01, может понадобиться NAT для сети 10.8.0.0/24

WireGuard поднимает туннель, но не отменяет обычные правила маршрутизации и фильтрации

WireGuard: peers, ключи и AllowedIPs

Как клиенты (peers) устанавливают защищённый туннель и какие сети они могут маршрутизировать

1 Что такое peer?



Peer (пир) — это удалённый узел WireGuard, с которым устанавливается VPN-туннель.

Каждый peer имеет свою пару криптографических ключей и обменивается публичными ключами для безопасного соединения.

4 Конфигурация клиента (laptop01)

```
[Interface]
Address = 10.8.0.2/24
PrivateKey = <LAPTOP01_PRIVATE_KEY>
DNS = 192.168.10.1

[Peer]
PublicKey = <SERVER_PUBLIC_KEY>
Endpoint = 203.0.113.10:51820
AllowedIPs = 10.8.0.0/24, 192.168.10.0/24
PersistentKeepalive = 25
```

- IP клиента в VPN-сети
- Публичный ключ сервера
- Публичный адрес сервера и порт WireGuard
- Какие сети доступны через VPN (вся VPN-сеть и корпоративная LAN)
- Поддержание соединения за NAT/фаерволом

5 AllowedIPs — что это и как работает

AllowedIPs определяет какие IP-сети «принадлежат» данному peer и будут отправляться в туннель.

Если сети нет в AllowedIPs — трафик пойдёт в обычный интернет, а не в туннель.

Пример маршрутизации на клиенте

Назначение	Куда отправить
10.8.0.0/24	через wg0 (VPN)
192.168.10.0/24	через wg0 (VPN)
8.8.8.8	через обычный интернет
0.0.0.0/0	через обычный интернет

Совет: указывайте только нужные сети, чтобы не туннелировать лишний трафик.



Кратко

- Каждый peer имеет ключи: **PrivateKey** (секрет) и **PublicKey** (для обмена).
- Peers обмениваются публичными ключами и устанавливают защищённый туннель.
- AllowedIPs** управляет тем, какие сети доступны через этот туннель.
- Комбинация `wg0 + ip_forward + NAT` позволяет безопасно подключаться к LAN из любой точки мира.

2 Криптографические ключи



Каждый узел генерирует пару ключей:

- PrivateKey** — приватный ключ (хранится в секрете)
- PublicKey** — публичный ключ (обменивается с peers)



однонаправленно



Приватный ключ никогда не передаётся по сети.

- Публичные ключи peers указываются в конфигурации.
- WireGuard использует современные алгоритмы (NoiseK, Curve25519, ChaCha20).

3 Конфигурация сервера (gw01)

```
[Interface]
Address = 10.8.0.1/24
ListenPort = 51820
PrivateKey = <SERVER_PRIVATE_KEY>
# Включаем маршрутизацию и NAT для LAN
PostUp = sysctl -w net.ipv4.ip_forward=1
PostUp = nft add rule inet filter forward iifname "wg0" oifname "enp0s8" accept
PostUp = nft add rule ip nat postrouting oifname "enp0s3" masquerade
PostDown = nft flush ruleset
```

```
[Peer]
PublicKey = <LAPTOP01_PUBLIC_KEY>
AllowedIPs = 10.8.0.2/32, 192.168.10.0/24
PersistentKeepalive = 25
```

6 Полезные команды

```
# Показать интерфейсы WireGuard
wg show

# Показать публичный ключ
wg show wg0 public-key

# Проверить маршрут к сети LAN
ip route get 192.168.10.20
```

```
# Пинг через VPN
ping 10.8.0.1

# Пинг узла в LAN через туннель
ping 192.168.10.20

# Логи ядра (при проблемах)
journalctl -k | grep wireguard
```



Важно

- Храните приватные ключи в безопасности.
- Используйте минимально необходимые сети в AllowedIPs.
- Регулярно обновляйте систему и следите за логами.



7. Практика: настройка WireGuard туннеля маршрутизатор-клиент

В учебном стенде gw01 будет WireGuard-сервером, а удаленный ноутбук будет клиентом

Сервер получает адрес 10.8.0.1/24 на интерфейсе wg0

Клиент получает адрес 10.8.0.2/32 внутри туннеля

Клиенту разрешается доступ к внутренней сети
192.168.10.0/24

Установка WireGuard и генерация ключей

`sudo apt install wireguard` -у устанавливает WireGuard и утилиты `wg`, `wg-quick`

`umask 077` ограничивает права на создаваемые файлы ключей

`wg genkey | tee server_private.key | wg pubkey > server_public.key` создает пару ключей сервера

Такую же пару ключей нужно создать для клиента и обменяться только открытыми ключами

Секция Interface сервера

[Interface] начинает настройки локального WireGuard-интерфейса

PrivateKey = <закрытый_ключ_сервера> указывает private key gw01

Address = 10.8.0.1/24 назначает адресу серверу внутри VPN

ListenPort = 51820 задает UDP-порт приема WireGuard-подключений

Секция Peer для клиента

[Peer] описывает удаленного клиента в конфигурации сервера

PublicKey = <открытый_ключ_клиента> указывает public key клиента

AllowedIPs = 10.8.0.2/32 закрепляет VPN-адрес клиента за его ключом

Если указать слишком широкие AllowedIPs, маршруты могут начать уходить не тому peer

Запуск и автозапуск wg0

`sudo wg-quick up wg0` поднимает интерфейс `wg0` по файлу `/etc/wireguard/wg0.conf`

`sudo wg` показывает состояние WireGuard, public key, endpoint и latest handshake

`sudo systemctl enable wg-quick@wg0` включает автозапуск туннеля при загрузке

`sudo wg-quick down wg0` корректно останавливает интерфейс

Маршрутизация VPN-клиента в LAN

На gw01 должен быть включен `net.ipv4.ip_forward=1`

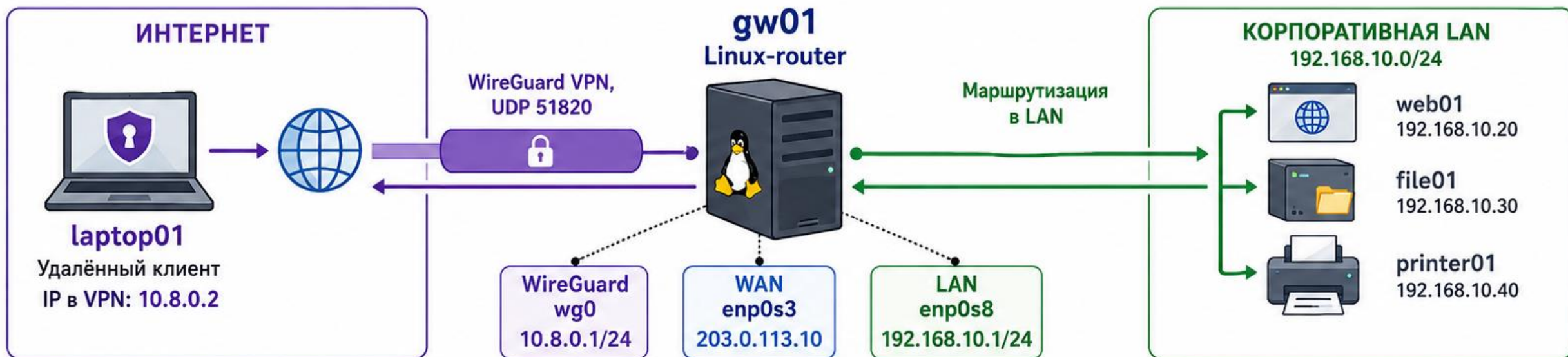
В nftables нужно разрешить forward между wg0 и enp0s8

Правило `sudo nft add rule inet filter forward iifname "wg0" oifname "enp0s8" accept` разрешает клиенту идти в LAN

Правило `established,related` разрешает обратные ответы из LAN в VPN-туннель

WireGuard remote client через gw01 в LAN

Как удалённый клиент подключается по WireGuard к Linux-router и получает доступ к корпоративной сети



1. КАК ЭТО РАБОТАЕТ

- 1 Клиент подключается к gw01 через Интернет.
- 2 Создаётся туннель WireGuard.
- 3 Клиент получает адрес 10.8.0.2.
- 4 gw01 маршрутизирует трафик из wg0 в LAN.
- 5 Ответы из LAN возвращаются клиенту через туннель.



2. ЧТО НУЖНО НА gw01

- IP forwarding включён.
- Разрешён UDP 51820.
- Forward между wg0 и enp0s8 разрешён.
- Есть маршрут к LAN.



3. ЧТО ПОЛУЧАЕТ КЛИЕНТ

- Доступ к 192.168.10.0/24.
- Работа как будто из офиса.
- Шифрование трафика.
- Доступ только к нужным сетям через AllowedIPs.



4. МИНИ-КОНФИГУРАЦИЯ

Сервер (gw01)

```
[Interface]
Address = 10.8.0.1/24
ListenPort = 51820

[Peer]
AllowedIPs = 10.8.0.2/32
```

Клиент (laptop01)

```
[Interface]
Address = 10.8.0.2/24

[Peer]
Endpoint = 203.0.113.10:51820
AllowedIPs = 192.168.10.0/24
```



5. ПРОВЕРКА

- 1 wg show
- 2 ip a
- 3 ping 10.8.0.1
- 4 ping 192.168.10.20



Ключевая идея: WireGuard создаёт защищённый туннель до gw01, а Linux-router маршрутизирует трафик из VPN в корпоративную LAN.

Защита файлов WireGuard

Файл `/etc/wireguard/wg0.conf` содержит закрытый ключ сервера и должен быть защищен
`sudo chmod 600 /etc/wireguard/wg0.conf` разрешает чтение и запись только владельцу

Закрытые ключи нельзя отправлять в чат, вставлять в отчеты или хранить в общей папке

При утечке `private key` нужно заменить ключи и обновить конфигурации `peer`

8. Диагностика, мониторинг и типичные ошибки

Диагностика начинается с проверки маршрутизации, затем firewall, затем NAT и VPN

Нужно отдельно проверять контрольный канал WireGuard и полезный трафик через туннель

Проблема может быть на сервере, клиенте, firewall, AllowedIPs или маршрутах

Для проверки лучше идти по уровням, а не сразу переписывать всю конфигурацию

Проверка Linux-router

`ip addr` показывает адреса интерфейсов gw01

`ip route` показывает маршруты и default route

`sysctl net.ipv4.ip_forward` проверяет, включена ли пересылка IPv4

`ping 192.168.10.1` с клиента LAN проверяет доступность внутреннего интерфейса маршрутизатора

Проверка nftables

`sudo nft list ruleset` показывает активные таблицы, цепочки и правила

Нужно проверить наличие filter-цепочки `forward` и правил `established,related`

Для NAT проверяют `postrouting masquerade` и `prerouting dnat`

Если `policy drop` включена, отсутствие одного асепт-правила полностью блокирует нужный поток

Проверка WireGuard

`sudo wg` показывает peer, endpoint, latest handshake и счетчики transfer

latest handshake появляется только после успешного обмена пакетами между пирами

`ping 10.8.0.1` с клиента проверяет доступность VPN-интерфейса сервера

`ping 192.168.10.20` с клиента проверяет маршрутизацию из VPN в LAN

Типовые ошибки WireGuard и маршрутизации

`ip_forward` выключен: туннель поднят, но LAN за сервером недоступна

`AllowedIPs` указаны неверно, поэтому маршрут уходит не в тот реер или не создается

UDP 51820 заблокирован `firewall` на внешнем интерфейсе

`Private key` и `public key` перепутаны при переносе в конфигурации

Forward-цепочка `nftables` запрещает трафик между `wg0` и `enp0s8`

Диагностика Linux-router, NAT и WireGuard

Пошаговая проверка маршрутизации, firewall, NAT и VPN в учебном стенде



1. Базовая проверка интерфейсов

- `ip a` — проверить адреса `enp0s3`, `enp0s8`, `wg0`
- `ip route` — проверить default route и локальные сети
- `ping 192.168.10.1` — доступность шлюза



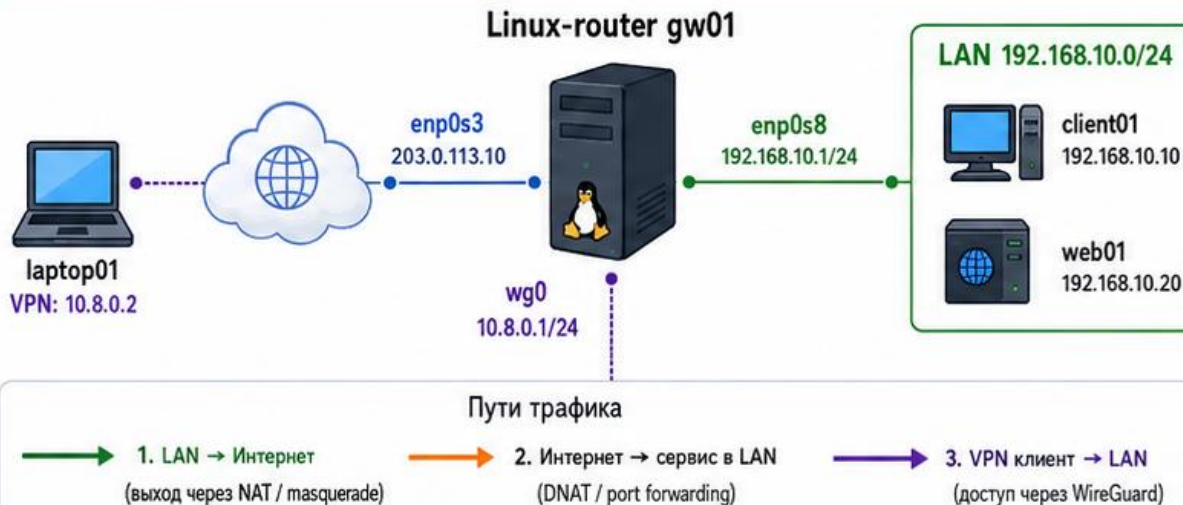
Если интерфейс down или адрес неверный, дальше проверять рано.



2. Проверка маршрутизации Linux-router

- `sysctl net.ipv4.ip_forward` → должно быть 1
- Проверить маршрут в Интернет через `enp0s3`
- Проверить маршрут к `192.168.10.0/24` и `10.8.0.0/24`

⚠ `ip_forward=0` — транзитный трафик не пойдёт



3. Проверка nftables / firewall

- `sudo nft list ruleset`
- Цепочка `input`: доступ к SSH / WireGuard / локальным службам
- Цепочка `forward`: разрешение LAN ↔ WAN и `wg0` ↔ LAN
- `policy drop` требует явных accept-правил

prerouting
(DNAT)

input
(к роутеру)

forward
(транзит)

postrouting
(NAT)



4. Проверка NAT и port forwarding

A) Masquerade для выхода LAN наружу



- правило masquerade в postrouting для oifname "enp0s3"
- клиент LAN должен иметь gateway 192.168.10.1

B) DNAT / Port Forwarding



- prerouting dnat 203.0.113.10:8080 -> 192.168.10.20:80
- forward должен разрешать трафик к web01

⚠ DNAT без forward не работает



5. Проверка WireGuard



- `wg show` — peer, latest handshake, transfer
- На gw01 разрешён UDP 51820
- AllowedIPs заданы корректно
- Должен быть разрешён forward между `wg0` и `enp0s8`



Туннель поднят, но LAN не видна? Проверить `ip_forward`, AllowedIPs и forward.

6. Практические тесты

- ✓ С client01: ping 8.8.8.8
- ✓ С client01: ping внешнего IP / проверка выхода через NAT
- ✓ С внешнего клиента: доступ к 203.0.113.10:8080
- ✓ С laptop01: ping 10.8.0.1
- ✓ С laptop01: ping 192.168.10.20
- ✓ traceroute / tracepath до нужного узла



7. Полезные команды

<code>\$ ip a</code>	<code>\$ sudo nft list ruleset</code>	<code>\$ traceroute 192.168.10.20</code>
<code>\$ ip route</code>	<code>\$ wg show</code>	<code>\$ sudo journalctl -u wg-quick@wg0</code>
<code>\$ sysctl net.ipv4.ip_forward</code>	<code>\$ ping 8.8.8.8</code>	<code>\$ sudo tcpdump -ni enp0s3</code>



Типовые симптомы и причины



- Нет Интернета у LAN
- gateway
 - `ip_forward`
 - masquerade
 - forward



- Порт 8080 снаружи не открывается
- DNAT
 - forward
 - сервис web01



- VPN handshake нет
- UDP 51820
 - ключи
 - endpoint



- VPN есть, но LAN не доступна
- AllowedIPs
 - forward `wg0` ↔ `enp0s8`
 - маршруты



Ключевая идея

Диагностика идёт по цепочке: интерфейсы -> маршруты -> `ip_forward` -> nftables -> NAT -> WireGuard -> тестовый трафик и логи.

Итоги лекции

Linux может быть маршрутизатором, если включен `ip_forward`, настроены маршруты и правила `forward` `nftables` управляет входящим, исходящим и транзитным трафиком через таблицы, цепочки и правила

`Masquerade` публикует внутреннюю сеть наружу, а `DNAT` публикует внутренний сервер через внешний порт

`WireGuard` строит быстрый VPN на ключах, `AllowedIPs` и UDP-порту 51820

Диагностика начинается с `ip route`, `sysctl`, `nft list ruleset`, `wg` и проверочных `ping`

Контрольные вопросы

Какая переменная ядра включает пересылку IPv4-пакетов между интерфейсами?

Чем nftables отличается от классического iptables на уровне архитектуры?

Что такое deny by default и почему этот подход важен для маршрутизатора?

Чем masquerade отличается от обычного SNAT?

На каком hook выполняется DNAT для port forwarding?

Контрольные вопросы: VPN и диагностика

Какие задачи решают Site-to-Site VPN и Remote Access VPN?

На каком транспортном протоколе и порту по умолчанию работает WireGuard?

Что означает Cryptokey Routing и как с ним связан AllowedIPs?

Какая команда показывает состояние WireGuard-туннеля?

Что проверить, если WireGuard-туннель поднят, но клиент не видит LAN?